◆ SENDING THE REQUEST TO FLASK

```
ResponseEntity<Map> response =
        restTemplate.postForEntity(
                flaskUrl,
                entity,
                Map.class
        );
```

This is the most important line in EmailService.

This is where Spring actually talks to Flask.

Everything before this line was preparation.

Now the request is sent.

What Is Happening Here?

Think of it like this:

```
Spring
   ↓
Creates Headers
   ↓
Creates JSON Body
   ↓
Builds Request
   ↓
Sends To Flask
```

This line performs the final step.

Let's Break It Down

```
restTemplate.postForEntity(...)
```

restTemplate

```
restTemplate
```

is the object we created earlier.

```
private final RestTemplate restTemplate =
        new RestTemplate();
```

Think of it as a messenger.

```
Spring
   ↓
RestTemplate
   ↓
Flask
```

postForEntity()

```
postForEntity(...)
```

means:

> "Send an HTTP POST request and return the response."

Think of it like:

```
Send Request
Wait
Receive Response
```

all in one method.

Why POST?

Your Flask endpoint is:

```
/predict
```

and usually accepts:

```
POST
```

requests.

POST is used when sending data to the server.

Example:

```
{
   "emailText":"You won a lottery"
}
```

Structure

```
postForEntity(
    URL,
    Request,
    ResponseType
)
```

Spring needs 3 things:

1. Where to send?
2. What to send?
3. What response type to expect?

◆ PARAMETER 1 : flaskUrl

```
flaskUrl
```

Remember:

```
@Value("${flask.api.url}")
private String flaskUrl;
```

Suppose:

```
flask.api.url=
http://localhost:5000/predict
```

Then:

```
flaskUrl
```

contains:

```
http://localhost:5000/predict
```

This tells Spring:

> "Send the request to Flask."

Visual

```
Spring
   ↓
http://localhost:5000/predict
   ↓
Flask
```

◆ PARAMETER 2 : entity

```
entity
```

contains:

```
Headers
+
Body
```

Previously we built:

```
HttpEntity<Map<String,String>>
```

Inside it:

```
Headers:
Content-Type: application/json
```

and

```
Body:
{
   "emailText":"You won a lottery"
}
```

So Spring sends:

```
POST /predict

Headers:
Content-Type: application/json

Body:
{
   "emailText":"You won a lottery"
}
```

◆ PARAMETER 3 : Map.class

```
Map.class   check sd pdf
```

This tells Spring:

> "I expect Flask to return JSON that can be stored inside a Map."

Why Map?

Suppose Flask returns:

```
{
   "prediction":"SPAM"
}
```

Spring converts this JSON into:

```
Map
```

Internally something like:

```
{
   prediction=SPAM
}
```

Now Spring can access values using keys.

Example:

```
map.get("prediction")
```

returns:

```
SPAM
```

Why Not Create Another DTO?

You could create:

```
PredictionResponseDTO
```

and store Flask response inside it.

But for a small response:

```
{
   "prediction":"SPAM"
}
```

using a Map is simpler.

◆ WHAT IS ResponseEntity<Map> ?

```
ResponseEntity<Map> response
```

Let's break this apart.

ResponseEntity

```
ResponseEntity
```

represents the complete response.

Contains:

```
Status Code
Headers
Body
```

Think of it as a box.

```
ResponseEntity
  ├── Status
  ├── Headers
  └── Body
```

<Map>

```
<Map>
```

means:

> "The body contains a Map."

So:

```
ResponseEntity<Map>
```

means:

```
Response
  ├── Status
```

```
 ├── Headers
 └── Map Body
```

Example

Flask returns:

```
HTTP 200 OK

{
   "prediction":"SPAM"
}
```

Spring stores:

```
response
```

containing:

```
Status = 200

Body = {
   prediction=SPAM
}
```

Visual

```
response
 ├── Status = 200
 └── Body
      └── prediction=SPAM
```

◆ WHAT HAPPENS INTERNALLY?

When this line executes:

```
restTemplate.postForEntity(...)
```

the following happens.

Step 1

Spring builds request.

```
Headers
+
Body
```

Step 2

Spring opens connection.

```
Spring
   ↓
localhost:5000
```

Step 3

Request reaches Flask.

```
POST /predict
```

Step 4

Flask receives:

```
{
   "emailText":"You won a lottery"
}
```

Step 5

Flask calls ML model.

```
Email Text
     ↓
ML Model
```

Step 6

Model predicts:

```
SPAM
```

Step 7

Flask creates response.

```
{
   "prediction":"SPAM"
}
```

Step 8

Response sent back.

```
Flask
   ↓
Spring
```

Step 9

Spring converts JSON into Map.

```
{
   "prediction":"SPAM"
}
```

becomes:

```
Map
```

Step 10

Spring stores everything inside:

```
response
```

Final result:

```
response
 ├── Status = 200
 └── Body
      └── prediction=SPAM
```

◆ WHY STORE RESPONSE IN A VARIABLE?

```
ResponseEntity<Map> response =
        restTemplate.postForEntity(...)
```

Because we need to access the returned data later.

Example:

```
response.getBody()
```

gets the body.

```
response.getStatusCode()
```

gets the status code.

```
response.getHeaders()
```

gets response headers.

So we save the response for future use.

◆ SUMMARY

```
restTemplate.postForEntity(...)
```

means:

> "Send a POST request to Flask and wait for the response."

```
flaskUrl
```

tells Spring where to send the request.

```
entity
```

contains headers and JSON body.

```
Map.class
```

tells Spring how to store the response.

```
ResponseEntity<Map>
```

stores the complete response returned by Flask.

After this line executes, Spring now has Flask's prediction and moves to:

```
String result = "ERROR";
```

followed by:

```
if (response.getBody() != null &&
    response.getBody().get("prediction") != null)
```

where it safely extracts the prediction value (`SPAM` or `HAM`) from the Flask response. This is the next section.